




Salma Mahmoud · You

 Computer Science Student at October 6 Univer...
 now ·

مش بيتغير؟ String فيه ال — C# في String Immutability

وأنا بتعلم C#, من الحاجات اللي وقفت معايا شوية هي فكرة إن ال String Immutable.

يعني إيه Immutable؟

ببساطة، يعني إن ال String بعد ما يتعمل، مينفعش نغير قيمته نفسها.

وده ممكن يكون غريب في الأول، لأننا بنكتب عادي جدًا:

```
string name = "Salma";
name = "Sara";
```

فممكن أسأل: "إزاي بتقولوا إن ال String مش بيتغير؟ أنا غيرته فعلاً!"

👉 هنا بقى الفكرة

اللي حصل إن ال String القديم "Salma" نفسه متغيرش.

ال "Salma" فضل زي ما هو، لكن لما كتبت:

```
name = "Sara";
```

اتعمل String جديد بقيمة "Sara"، وال variable name بقى بيشاور عليه بدل القديم.

وده لأن ال String في C# immutable.

طيب ليه اتعمل بالشكل ده؟

ال Immutability ليه مميزات مهمة، منها إنها بتخلي التعامل مع ال Strings أكثر أمانًا، خصوصًا لما نفس ال String يكون مستخدم في أكثر من مكان.



Home



My Network



Post



Notifications



Jobs

وكمان بتخلي ال Strings مناسبة للاستخدام في حاجات زي ال Hashing وال Dictionary keys، لأن القيمة مش هتتغير فجأة بعد ما يتم استخدامها.

طب هل ده معناه إن كل تعديل على String بيعمل Object جديد؟

أيوه، ودي نقطة مهمة جدًا.

لو عملنا مثلاً:

```
string text = "Hello";
text += " World";
text += "!";
```

كل عملية تعديل هنا بتنتج String جديد.

وده مش بيكون مشكلة لو التعديلات قليلة.

لكن تخيل إن عندي Loop وبتعامل مع String كبير، ويعمل عليه تعديلات مئات أو آلاف المرات.

هنا الموضوع ممكن يبقى أقل كفاءة، وده أحد الأسباب اللي بنستخدم فيها StringBuilder.

```
StringBuilder text = new StringBuilder("Hello");
text.Append(" World");
text.Append("!");
```

ال StringBuilder معمول عشان نتعامل بكفاءة أكبر مع التعديلات المتكررة على النص.

الخلاصة

ال String في C# مش بيتغير من الداخل.

لما أقول:

```
text = "New Value";
```

أنا مش بغير ال String القديم، أنا بخلي ال variable يشاور على String جديد.

ودي من الأفكار البسيطة في C#، لكن فهمها بيساعد جدًا في فهم ال memory وال performance بشكل أعمق.

Sometimes, understanding why something works is more

لما أقول:

```
text = "New Value";
```

أنا مش بغير ال String القديم، أنا بخلي ال variable يشاور على String جديد.

ودي من الأفكار البسيطة في C#، لكن فهمها بيساعد جدًا في فهم ال memory وال performance بشكل أعمق.

Sometimes, understanding why something works is more important than just knowing how to use it.

#CSharp #DotNet #Programming #String
#StringBuilder #SoftwareDevelopment

Show translation

String Immutability

ليه ال String مش بيتغير؟

يعني إيه Immutable؟

يعني إن ال String بعد ما يتمعمل، مينفعش تغير قيمته نفسها.

إزاي ده يشتغل؟

لما نعمل كده:

```
string name = "Salma";
name = "Sara";
```

اللي حصل إن "Salma" فضل زي ما هو، والتعمل String جديد "Sara"، وال variable name يبقى يشاور عليه بدل القديم.

ليه تعمل بالشكل ده؟

أكثر أمانًا
خاصة لما نفس ال String
يكون مستخدم في أكثر
من مكان.

مناسب للهياكل للمهمة
و Dictionary و Hashing و زي
لأن القيمة مش هتغير فجأة بعد ما
بنم استخدامها.

**كل تعديل على String
يعمل Object جديد**

```
string text = "Hello";
text += " World";
text += "!";
```

كل عملية تعديل هنا بتنتج String جديد
ده ممكن يكون أقل كفاءة لو التعديلات
كبيرة أو النص كبير.

**الحل الأفضل للتعديلات الكثيرة
استخدم StringBuilder**

```
var text = new StringBuilder("Hello");
text.Append(" World");
text.Append("!");
```

StringBuilder معمول عشان نتعامل
بكفاءة أكبر مع التعديلات المتكررة.

Object 1: "Hello" → Object 2: "Hello World" → Object 3: "Hello World!"

StringBuilder: "Hello" (نفس الكائن) → "Hello World" (نفس الكائن) → "Hello World!" (نفس الكائن)

الخلاصة

ال String في C# مش بيتغير من الداخل.
أنا أقول: `text = "New Value";`
أنا مش بغير ال String القديم.
أنا بخلي ال variable يشاور على String جديد.

“ Sometimes, understanding why something works is more important than just knowing how to use it.”

Questions:

Q1:

`int.Parse(null)` throws an `ArgumentNullException`,
while `Convert.ToInt32(null)` returns 0.

Q2:

`TryParse` is recommended because it doesn't throw an exception when the input is invalid.
It returns false, allowing the application to handle invalid user input safely and continue running.

Q3:

`GetHashCode()` returns a hash code for an object, mainly used by hash-based collections
to quickly find and compare objects.

Q4:

Reference equality checks whether two variables refer to the exact same object in memory,
rather than whether their values are equal.

Q5:

Strings are immutable in C# for security, performance, and thread safety.
Once a string is created, its value cannot be changed; any modification creates a new string.

Q6:

`StringBuilder` is more efficient because it changes the same string buffer instead of creating a new string every time,
especially when we need to concatenate many strings.

Q7:

Because `StringBuilder` modifies the same buffer instead of creating a new string every time,
so it uses less memory and is faster when making many changes to a large string.

Q8:

String interpolation is the most commonly used because it is simple, readable, and easy to write,
especially when we need to include variables inside a string.

Q9:

`StringBuilder` is designed to modify the same internal buffer instead of creating a new string every time.
So, when we make frequent changes,
it is faster and uses less memory than regular strings.

part02:

Q2:

an enum is a data type used to define a set of named constant values.

It is useful when a variable can have only a specific set of options.

three common built-in enums are DayOfWeek, ConsoleColor, and StringComparison.

Q3:

use string when you don't need to change the text often.

use StringBuilder when you need to make many changes or concatenations,

especially with large strings.

Bouns:

Q5:

A user-defined constructor is a constructor that

we create ourselves to initialize an object with specific values when it is created.

Q6:

An Array stores elements in a fixed-size, continuous memory location,

while a Linked List stores elements in separate nodes connected by links. Arrays are faster for accessing elements,

while Linked Lists are better for frequent insertion and deletion.